

Diagram Layout with ELK

A Nasdanika Zephyr Book

Pavel Vlasov

Version 2026.11.0, 2026-11-07

Table of Contents

Colophon . . . 1

Preface . . . 3

Who this book is for . . . 3

A book that stands on its own . . . 3

What you need . . . 4

Conventions . . . 4

I: The Why . . . 5

1. Why diagrams need automatic layout . . . 6

1.1. Diagrams nobody drew . . . 6

1.2. Layout belongs to the model, not to the editor . . . 6

1.3. What a layout can and cannot do . . . 7

2. The Eclipse Layout Kernel, and an adapter . . . 8

2.1. ELK . . . 8

2.2. An adapter, not an abstraction . . . 8

2.3. What the module is . . . 9

II: The How . . . 10

3. Getting started . . . 11

3.1. Adding the module . . . 11

3.2. The diagram model in brief . . . 12

4. Laying out a diagram . . . 13

4.1. A pipeline . . . 13

4.2. What maps to what . . . 14

4.3. Coordinates . . . 14

4.4. Ports . . . 15

4.5. Nesting . . . 16

4.6. Options . . . 17

4.7. Sizes of text . . . 17

4.8. Layout as a unit of work . . . 18

5. Diagrams of Ecore models . . . 19

5.1. The conventions . . . 19

5.2. Options for class diagrams . . . 19

5.3. Ports for generalizations . . . 20

5.4. References, and their opposites . . . 21

5.5. The package diagram . . . 22

5.6. The class context diagram . . . 24
5.7. Checking the layout . . . 26
6. Rendering with React Flow and Draw.io . . . 29
6.1. React Flow . . . 29
6.2. Draw.io . . . 32
6.3. The site, and serving it . . . 34
7. Figures of generated diagrams . . . 37
III: Looking Ahead . . . 39
8. What comes next . . . 40
8.1. Renderers as modules . . . 40
8.2. Laying out what people drew . . . 40
8.3. Layout in the browser . . . 40
8.4. A layout service, when there is a choice . . . 40
8.5. Labels measured, not estimated . . . 41
8.6. Diagrams in narrated videos . . . 41
Appendix A: Layout options used most . . . 42
Appendix B: Building this book, and its figures . . . 44
B.1. The tests run on the module path . . . 44
Appendix C: Glossary . . . 45

Colophon

Diagram Layout with ELK: A Nasdanika Zephyr Book

Edition 2026.11.0, built on 2026-11-07.

This edition describes the ELK module of the diagram model of the Nasdanika SDK Runtime 2026.11.0, Maven coordinates `org.nasdanika.sdk.runtime.models.diagram:elk:2026.11.0`, with the Eclipse Layout Kernel 0.12.0. Every listing in it is included from the module's sources or tests, which are compiled and run when the book is built. Its figures are diagrams generated and laid out by those tests, rendered with Draw.io's viewer into PNG images with the Draw.io document embedded.

Copyright © 2026 Pavel Vlasov.

The text and figures of this book are licensed under the Creative Commons Attribution 4.0 International License (CC BY 4.0). You may share and adapt them for any purpose, commercial purposes included, provided you give appropriate credit, provide a link to the license, and indicate if you made changes. A suggested credit: "Pavel Vlasov, *Diagram Layout with ELK: A Nasdanika Zephyr Book*, licensed under CC BY 4.0", with a link to github.com/Nasdanika-SDK/runtime.

The code is licensed under the Eclipse Public License 2.0, the license of the runtime and of ELK. In addition, you may use the code of this book's listings in your own programs and documentation without asking for permission and without attribution.

Neither license covers the Nasdanika name or logo: CC BY 4.0 grants no trademark rights.

Eclipse, EMF and ELK are trademarks of the Eclipse Foundation. draw.io is a trademark of JGraph Ltd. React is a trademark of Meta Platforms, Inc. React Flow is a product of xyflow. Java is a trademark of Oracle and its affiliates. Other names may be trademarks of their owners. Their use here identifies the technologies the module works with and implies no endorsement.

The source of this book, its tests and the module it describes are at github.com/Nasdanika-SDK/runtime, in `models/diagram/elk`.

Typeset with Asciidoctor PDF in Noto Sans and M+ 1mn, in the look of the Nasdanika Zephyr

Books.

Preface

This book is about placing boxes and routing lines so that a person can read a diagram that nobody drew. It describes a small module of the Nasdanika SDK Runtime that lays out diagrams of the runtime's diagram model with the Eclipse Layout Kernel, ELK, and a set of demos that generate diagrams of Ecore models, lay them out, and render them with React Flow in a browser and with Draw.io.

The first part is the why: why generated diagrams need automatic layout, why ELK, and why the module is an adapter rather than an abstraction. The second part is the how. The third says what comes next.

Who this book is for

Developers who generate diagrams, from models, code, infrastructure or data, and need them to be readable without anyone moving a box.

People who build diagram tools and want ELK's layouts in a Java pipeline, over a neutral diagram model rather than the format of one editor.

Architects who want to know what automatic layout can and cannot do for a diagram estate. Part one is for you, and needs no code.

A book that stands on its own

This book is one of the Nasdanika Zephyr Books: in-depth manuals, one per module of the Nasdanika SDK Runtime, in the manner of IBM Redbooks. The diagram model has a Zephyr Book of its own, *Diagram Model*, and so does the runtime's Draw.io module, *Draw.io*. You do not need either to read this one: the few facts about the diagram model that layout depends on are repeated here.

The series is named after the Bootswatch theme it is styled with, Zephyr, and has nothing to do with the Zephyr real-time operating system or with SmartBear's Zephyr test management.

What you need

- **A JDK for Java 25** and **Maven 3.9.16 or later**.
- The module is `org.nasdanika.sdk.runtime.models.diagram.elk`, Maven coordinates `org.nasdanika.sdk.runtime.models.diagram:elk:2026.11.0`.
- For the demos, a browser with internet access: the pages load React, React Flow and Draw.io's viewer from content delivery networks.

Conventions

- **Every listing is included from the module's sources or tests** and compiled and run when the book is built. Callouts, such as <1>, explain the lines they mark.
- **The figures are generated.** The tests generate the diagrams, lay them out and write them as Draw.io documents; the figures are those documents rendered with Draw.io's viewer, with the document embedded in the image (Chapter 7).
- **Status labels** mark what this edition does not have yet: PLANNED and SKETCH.
- **Voice.** I write in the first person.

Part I: The Why

Why diagrams need automatic layout, why the Eclipse Layout Kernel, and why this module is an adapter rather than an abstraction. No code is needed to read this part.

Chapter 1. Why diagrams need automatic layout

A well laid out diagram compresses hours of reading into seconds of comprehension. A badly laid out one, with lines crossing for no reason, boxes overlapping, and related things far apart, costs more to read than the text it replaces. People who draw diagrams by hand spend much of their time on layout, and they are right to.

Programs do not have that time. And more and more diagrams are drawn by programs.

1.1. Diagrams nobody drew

- **Diagrams generated from models**: the classes of a metamodel, the modules of a running JVM, the services of a deployment, the tables of a database. The model is the truth; the diagram is a view of it, generated again whenever the model changes.
- **Diagrams converted between notations**: a Visio drawing loaded into a neutral model and written as a Draw.io document keeps its positions, but a diagram assembled from several sources has none.
- **Diagrams merged from versions**: a diagram regenerated from its source, merged with the edits people made to the previous generation, has new elements with no place yet.
- **Diagrams written by assistants**: a language model can say what is connected to what far more reliably than where each box should go.

In each case the structure is known and the geometry is not. That is exactly the problem automatic layout solves: given nodes, their sizes, their nesting and their connections, find positions and routes that a person can read.

1.2. Layout belongs to the model, not to the editor

Editors have layouts of their own. Draw.io arranges selections in trees, circles and grids; Visio and yEd have more. But a layout in an editor runs when a person asks for it, in that editor, on that editor's format. A diagram generated in a build has no editor and no person.

The runtime's diagram model is the hub between notations: Draw.io, PowerPoint and generated

diagrams all pass through it. Laying out the model lays out every notation that loads into it and every notation it is written to, once. This module does that.

1.3. What a layout can and cannot do

Automatic layout is good at what is mechanical: no overlaps, consistent spacing, edges that flow in one direction, few crossings, orthogonal routes that read like circuit diagrams, nested elements kept inside their containers.

It knows nothing about meaning. It does not know that the customer should be at the top, that two systems belong side by side because the same team owns them, or that one connection matters more than the rest. Those decisions are what people add to a generated diagram, and why a generated diagram that people improve should be merged rather than regenerated from scratch. Layout options and the order of elements let a generator express some of that intent; the rest stays with people.

Chapter 2. The Eclipse Layout Kernel, and an adapter

2.1. ELK

The Eclipse Layout Kernel (eclipse.dev/elk) is a collection of layout algorithms for node-link diagrams, developed at Kiel University and published under the Eclipse Public License 2.0.

- **Algorithms.** Layered, for directed graphs and flows, the Sugiyama approach with ports, labels and orthogonal routing; tree; force-directed and stress; radial; rectangle packing; and more.
- **One vocabulary of options.** Several hundred layout options, each with an id such as `elk.direction` or `elk.edgeRouting`, a type and a default, set on the graph, on a node, an edge, a port or a label.
- **Hierarchy.** Nodes contain nodes, and edges between nodes in different containers can be laid out across the hierarchy.
- **A graph model in EMF.** ELK's graph, `ElkNode`, `ElkPort`, `ElkEdge`, `ElkLabel`, is an EMF model, as the runtime's diagram model is.
- **The same layouts in the browser.** elkjs is ELK compiled to JavaScript, with the same algorithms and options, and React Flow documents it among its layout options.

Eclipse Sirius, Sprotty and the Graphical Language Server Platform use it, and in my experience it is the most capable open-source layout library on the JVM.

2.2. An adapter, not an abstraction

The module could have started with a layout service: an interface in the diagram model, implementations found through the runtime's capability framework, and a requirement object that names an engine and its options. It does not, on purpose.

- **There is one engine.** An interface designed around a single implementation is that implementation's shape with another name. Its options would be ELK's, renamed.
- **ELK is already the abstraction.** A dozen algorithms behind one API and one vocabulary of options. A wrapper could only narrow it.

- **Nothing needs to choose yet.** The capability framework is for choosing among implementations by requirement. The other layout engines a diagram meets, Draw.io's own and elkjs, run in JavaScript, not in the JVM.

So the module is an adapter: from the diagram model to ELK's graph, ELK's options from the diagram's own attributes, and the results written back into the diagram. Its entry point, `ElkLayout.layout(DiagramElement)`, is already the shape a layout service would have. When a second engine has a real consumer, such as a command that lets a user pick one, that signature becomes an interface and ELK one capability providing it; the adapter does not change.

2.3. What the module is

- `ElkLayout`: configuration, and `layout(DiagramElement)`, which builds the ELK graph, lays it out in a telemetry span, and writes the results back.
- `DiagramElkGraph`: the ELK graph of a diagram, with the correspondence between the two in both directions, so that code can inspect or adjust the ELK graph before the layout and the results after.
- `TextMeasurer`: the size of text, for nodes without bounds and for labels.

Everything else in this book, the generators of Ecore diagrams, the React Flow and Draw.io renderers and the site, is in the module's tests: demos, and the listings of the book.

Part II: The How

The adapter in use: adding it, laying out diagrams, the mapping and its coordinates, diagrams of Ecore models generated for layout, and the laid out diagrams rendered with React Flow and Draw.io and served over HTTP.

Chapter 3. Getting started

3.1. Adding the module

Depend on `org.nasdanika.sdk.runtime.models.diagram:elk` with the version managed by the runtime's bill of materials, `org.nasdanika.sdk.runtime:bom`. The module brings the diagram model, ELK's core and graph, and four algorithms: layered, tree (mrtree), force and radial. The BOM pins them all to ELK 0.12.0, and pins Guava, which ELK asks for with an open version range that would otherwise resolve to whatever is newest.

Listing 1. The module declaration

```
/**
 * Layout of the diagram model with the Eclipse Layout Kernel.
 * The ELK jars are automatic modules. Each algorithm registers itself with
 * ELK's layout meta data service through META-INF/services, so the
 * algorithms only have to be resolved, which the requires below make sure of.
 */
module org.nasdanika.sdk.runtime.models.diagram.elk {

    exports org.nasdanika.sdk.runtime.models.diagram.elk;

    requires transitive org.nasdanika.sdk.runtime.models.diagram;
    requires org.nasdanika.sdk.runtime.common; // Telemetry
    requires transitive org.eclipse.elk.core;
    requires transitive org.eclipse.elk.graph;
    requires org.eclipse.elk.alg.layered;
    requires org.eclipse.elk.alg.mrtree;
    requires org.eclipse.elk.alg.force;
    requires org.eclipse.elk.alg.radial;

}
```

ELK's jars are not Java modules; they are automatic modules, named in their manifests. Each algorithm registers itself with ELK's meta data service through `META-INF/services`, which works for automatic modules on the module path, as long as they are resolved: hence the `requires` of the algorithms, which the module's code never calls. To add another algorithm, rectangle packing for example, depend on it and require it in your own module.

3.2. The diagram model in brief

Layout works on the runtime's diagram model, an EMF model of diagrams independent of any editor:

- a **diagram** contains diagram elements, and diagrams nest, as documents contain pages;
- a **node** has **bounds**, a position and a size, relative to the node that contains it, and **ports**, attachment points at a location relative to the node's size, from 0 to 1;
- a **connection** runs from a **source** to a **target**, either a node or a port, through **waypoints**;
- a **label** is a node whose content is text; labels of a connection are its children;
- every element has an id, a name and **attributes**, a map from strings to strings.

Attributes are where layout options go (Chapter 4).

Chapter 4. Laying out a diagram

4.1. A pipeline

Three nodes and two connections, laid out left to right:

Listing 2. Laying out a pipeline (`LayoutTests.java`)

```
Diagram diagram = FACTORY.createDiagram();
diagram.setId("pipeline");
diagram.getAttributes().put("elk.direction", "RIGHT"); ①
Node build = node(diagram, "Build");
Node test = node(diagram, "Test");
Node deploy = node(diagram, "Deploy");
connect(diagram, "artifacts", build, test);
Connection release = connect(diagram, "release", test, deploy);

new ElkLayout().layout(diagram); ②

Bounds b = build.getBounds();
Bounds t = test.getBounds();
Bounds d = deploy.getBounds();
assertThat(b.getX() + b.getWidth()).isLessThan(t.getX()); ③
assertThat(t.getX() + t.getWidth()).isLessThan(d.getX());
assertThat(b.getWidth()).isEqualTo(80); ④

List<Point> route = release.getWaypoints(); ⑤
assertThat(route.getFirst().getX()).isEqualTo(t.getX() + t.getWidth());
assertThat(route.getLast().getX()).isEqualTo(d.getX());
Label label = (Label) release.getChildren().getFirst(); ⑥
assertThat(label.getText()).isEqualTo("release");
assertThat(label.getBounds().getWidth()).isPositive();
```

① A layout option, as an attribute of the diagram. Keys that start with `elk.` or `org.eclipse.elk.` are ELK options; the value is parsed by the option's type.

② Builds the ELK graph, lays it out, and writes the results back into the diagram.

③ The nodes now have bounds, in the order of the flow.

④ A node without bounds is sized to fit its name, here the minimum width.

⑤ The waypoints are the whole route, from the right side of *Test* to the left side of *Deploy*.

⑥ A connection without labels gets a label for its name, so that ELK can place it and its position has somewhere to go.

4.2. What maps to what

Diagram model	ELK	Written back
The element laid out, usually a diagram	The root `ElkNode`	Its size, if it is a node
Node, and label standing alone	`ElkNode`, nested as the diagram nests them	Bounds, relative to the nearest enclosing node
Diagram and layer inside the root	Transparent: their nodes join the enclosing ELK node	Nothing
Port	`ElkPort` at its location, on the side the location is on	Location, relative to the node's size
Connection	`ElkEdge` between the ELK nodes and ports of its ends	Waypoints: the whole route
Label child of a connection, or the connection's name	`ElkLabel` of the edge	Bounds, in the coordinates of the waypoints
Attributes with `elk.` keys	Layout options of the element	Nothing

Connections with a missing end are not laid out; `DiagramElkGraph.getSkipped()` lists them.

4.3. Coordinates

Bounds of a node are relative to the nearest node that contains it, or to the diagram: the convention of Draw.io and of ELK. Waypoints and the bounds of connection labels are relative to the nearest node that contains the **connection**, or to the diagram. ELK may route an edge in a different container than the diagram puts the connection in, at the lowest node that contains both of its ends; the adapter converts ELK's coordinates into the connection's.

After a layout, the waypoints are the complete route, including the points where it leaves the source and reaches the target. A renderer can draw them as they are, which is what the renderers of Chapter 6 do. A renderer that computes ends itself, as Draw.io does, can use the first and last points as attachment points and the rest as waypoints.

4.4. Ports

A port's location decides its side: 0 on y is the top, 1 the bottom, 0 on x the left, 1 the right. Nodes with ports get ELK's `FIXED_POS` port constraint, so the ports stay where the diagram put them, and connections start and end exactly there:

Listing 3. Connections between ports

```
Diagram diagram = FACTORY.createDiagram();
diagram.setId("ports");
diagram.getAttributes().put("elk.direction", "UP");
diagram.getAttributes().put("elk.edgeRouting", "ORTHOGONAL");
Node general = node(diagram, "General");
Node special = node(diagram, "Special");
Port top = FACTORY.createPort(); ①
top.setId("top");
top.setLocation(point(0.5, 0));
special.getPorts().add(top);
Port bottom = FACTORY.createPort();
bottom.setId("bottom");
bottom.setLocation(point(0.5, 1));
general.getPorts().add(bottom);
Connection extension = connect(diagram, null, top, bottom);

new ElkLayout().layout(diagram);

Bounds g = general.getBounds();
Bounds s = special.getBounds();
assertThat(g.getY() + g.getHeight()).isLessThan(s.getY()); ②
Point start = extension.getWaypoints().getFirst();
Point end = extension.getWaypoints().getLast();
assertThat(start.getX()).isEqualTo(s.getX() + s.getWidth() / 2); ③
assertThat(start.getY()).isEqualTo(s.getY());
assertThat(end.getX()).isEqualTo(g.getX() + g.getWidth() / 2);
assertThat(end.getY()).isEqualTo(g.getY() + g.getHeight());
assertThat(extension.getChildren()).isEmpty(); ④
```

① A port at the middle of the top side of *Special*, and another at the middle of the bottom side of *General*.

② With the direction up, the target of the connection is above its source.

③ The route starts exactly at the port.

④ A connection without a name gets no label.

4.5. Nesting

Nodes contain nodes. The adapter lays out the whole hierarchy at once, so that connections between nodes in different containers are routed across it:

Listing 4. Nodes inside nodes, and a connection into a container

```
Diagram diagram = FACTORY.createDiagram();
diagram.setId("nested");
Node system = node(diagram, "System");
Node api = FACTORY.createNode();
api.setId("API");
api.setName("API");
system.getChildren().add(api); ①
Node database = FACTORY.createNode();
database.setId("Database");
database.setName("Database");
system.getChildren().add(database);
Node client = node(diagram, "Client");
Connection calls = connect(diagram, "calls", client, api); ②
system.getChildren().add(connect(diagram, "reads", api, database));

DiagramElkGraph graph = new ElkLayout().layout(diagram);

Bounds s = system.getBounds();
Bounds a = api.getBounds();
assertThat(a.getX()).isPositive(); ③
assertThat(a.getX() + a.getWidth()).isLessThanOrEqualTo(s.getWidth());
ElkEdge elkCalls = (ElkEdge) graph.getElkElement(calls);
assertThat(elkCalls.getContainingNode()).isSameAs(graph.getElkRoot()); ④
Point end = calls.getWaypoints().getLast();
double apiLeft = s.getX() + a.getX();
assertThat(end.getX()).isBetween(apiLeft, apiLeft + a.getWidth()); ⑤
```

① The API and the database are inside the system.

② The client, outside, calls the API, inside.

③ Bounds of nested nodes are relative to their container, with room for its name at the top.

④ ELK routes the edge in the root, the lowest node containing both ends.

⑤ The route ends on the API, in the coordinates of the diagram, where the connection is.

When any node has children, the adapter sets ELK's `elk.hierarchyHandling` to `INCLUDE_CHILDREN` on the root, unless the root's attributes say otherwise. Containers get padding at the top for their name, and a minimum size to fit it.

4.6. Options

Options come from three places, later ones winning: ELK's defaults, options set on `ElkLayout` in code, which apply to the root, and the attributes of diagram elements.

Listing 5. Options in code and in attributes

```
Diagram diagram = FACTORY.createDiagram();
diagram.setId("options");
node(diagram, "A").getAttributes().put("elk.nodeSize.minimum", "(200, 100)");

DiagramElkGraph graph = new ElkLayout()
    .option(CoreOptions.DIRECTION, Direction.DOWN) ①
    .build(diagram); ②
ElkNode elkRoot = graph.getElkRoot();
assertThat(elkRoot.getProperty(CoreOptions.DIRECTION)).isEqualTo(Direction.DOWN);
ElkNode a = elkRoot.getChildren().getFirst();
assertThat(a.getProperty(CoreOptions.NODE_SIZE_MINIMUM).x).isEqualTo(200);

diagram.getAttributes().put("elk.direction", "SIDEWAYS");
assertThatThrownBy(() -> new ElkLayout().build(diagram)) ③
    .isInstanceOf(IllegalArgumentException.class)
    .hasMessageContaining("elk.direction");
```

① An option of the root, typed: an ELK `IProperty` and its value.

② `build` makes the ELK graph without laying it out, to inspect it or to change it before `layout()` and `apply()`.

③ An unknown option, or a value the option cannot parse, fails before anything is laid out.

Options in attributes are data: they travel with the diagram, in a file, in a model, from a generator, and an editor that keeps unknown attributes keeps them. Options in code suit what a program decides for every diagram it lays out. Appendix A lists the ones used most.

4.7. Sizes of text

ELK reserves room for labels by their size, and the adapter sizes nodes without bounds by their name. Both use a `TextMeasurer`, which by default estimates 7 pixels per character and 16 per line, close to 12-pixel sans serif text in a browser. A renderer that uses another font should use another measurer, in the configuration: `new ElkLayout().textMeasurer(…)`. Padding around names and the minimum node size are configured the same way.

4.8. Layout as a unit of work

Listing 6. The entry point (`ElkLayout.java`)

```
/**
 * Builds the ELK graph of a diagram element, lays it out, and writes the results back into the
 * diagram, in a span of the current OpenTelemetry instance.
 */
public DiagramElkGraph layout(DiagramElement root) {
    DiagramElkGraph graph = build(root);
    Attributes attributes = Attributes.of(
            ROOT, String.valueOf(root.getId()),
            NODES, count(graph.getElkRoot()),
            EDGES, (long) graph.getElkRoot().getContainedEdges().size());
    Telemetry.inSpan(Telemetry.tracer(Telemetry.current()), "ELK Layout", attributes, span -> {
        graph.layout(new BasicProgressMonitor());
        graph.apply();
        return null;
    });
    return graph;
}
```

A layout runs in a span of the current OpenTelemetry instance, with the root's id and the numbers of nodes and edges as attributes, so a slow layout shows up in the runtime's telemetry like any other unit of work.

Chapter 5. Diagrams of Ecore models

The demos generate two kinds of class diagram from Ecore models, in the diagram model, and lay them out:

- a **package diagram**: the classes of a package, and the classes of other packages that they extend or refer to;
- a **class context diagram**: a class, all its supertypes, its direct subtypes, the classes its references lead to, and the classes whose references lead to it.

The model they are drawn from is the diagram model itself.

5.1. The conventions

The generator, `EcoreDiagrams`, follows a few conventions, which the layout and the renderers rely on:

- **Classes are nodes inside nodes of their packages**, so the classes of other packages appear in their own containers.
- **Generalizations are rectilinear, from a port at the top of the subclass to a port at the bottom of the superclass.** Every generalization into the same superclass arrives at the same port, so they merge into one line, as UML draws them.
- **References connect nodes, not ports**, and ELK decides where they leave and arrive.
- **A reference and its opposite are one connection**, with a label at each end naming the reference that end leads to. Any other reference has one label, at its center.
- **Each element depicts what it stands for**: a class node depicts its `EClass`, a connection its `EReference` or the pair, through `semanticElements`.

5.2. Options for class diagrams

The generator sets the layout options of its diagrams as attributes:

Listing 7. Layout options of the generated diagrams (`EcoreDiagrams.java`)

```
var options = diagram.getAttributes();
options.put("elk.algorithm", "layered");
```

```
options.put("elk.direction", "UP"); ①
options.put("elk.edgeRouting", "ORTHOGONAL"); ②
options.put("elk.hierarchyHandling", "INCLUDE_CHILDREN"); ③
options.put("elk.layered.spacing.nodeNodeBetweenLayers", "50");
options.put("elk.spacing.nodeNode", "40");
options.put("elk.spacing.edgeLabel", "4");
options.put("elk.layered.mergeEdges", "true"); ④
```

① Edges flow up: from subclasses to superclasses, from the class that has a reference to the class it refers to.

② Rectilinear routes, with horizontal and vertical segments only.

③ Classes in packages, laid out together, so that connections between packages are routed across them.

④ Edges into the same port share their last segments, which merges generalizations into one superclass.

5.3. Ports for generalizations

Listing 8. A port at the middle of the top or bottom side

```
private Port port(Node node, String id, double y) {
    for (Port port: node.getPorts()) {
        if (id.equals(port.getId())) {
            return port;
        }
    }
    Port port = factory.createPort();
    port.setId(id);
    Point location = factory.createPoint();
    location.setX(0.5);
    location.setY(y); ①
    port.setLocation(location);
    node.getPorts().add(port);
    return port;
}
```

① 0 for the top, 1 for the bottom; the x coordinate is always the middle.

Listing 9. Generalizations, from port to port

```
private void generalizations() {
    for (Map.Entry<EClass, Node> entry: classes.entrySet()) {
        for (EClass superType: entry.getKey().getESuperTypes()) {
            Node superNode = classes.get(superType);
            if (superNode != null) {
                Connection connection = factory.createConnection();
```

```
                connection.setId(entry.getKey().getName() + "-extends-" + superType.getName());
                connection.setSource(port(entry.getValue(), TOP, 0)); ①
                connection.setTarget(port(superNode, BOTTOM, 1)); ②
                connection.getAttributes().put(KIND, GENERALIZATION);
                diagram.getChildren().add(connection);
            }
        }
    }
}
```

① The source is the port at the top of the subclass.

② The target is the port at the bottom of the superclass, shared by all its subclasses.

5.4. References, and their opposites

Listing 10. References: one connection per reference, or per pair of opposites

```
private void references(Collection<EClass> sources, Predicate<EReference> exclude) {
    Set<EReference> done = new HashSet<>();
    for (EClass source: sources) {
        for (EReference reference: source.getEReferences()) {
            Node target = classes.get(reference.getEReferenceType());
            if (target == null || done.contains(reference) || exclude.test(reference)) {
                continue;
            }
            EReference opposite = reference.getEOpposite();
            Connection connection = factory.createConnection();
            connection.setId(source.getName() + "." + reference.getName());
            connection.setSource(classes.get(source)); ①
            connection.setTarget(target);
            connection.getSemanticElements().add(reference);
            connection.getAttributes().put(KIND, REFERENCE);
            if (reference.isContainment()) {
                connection.getAttributes().put(CONTAINMENT, "true");
            }
            done.add(reference);
            if (opposite == null) {
                connection.getChildren().add(label(reference, "CENTER")); ②
            } else {
                connection.getSemanticElements().add(opposite);
                if (opposite.isContainment()) {
                    connection.getAttributes().put(CONTAINER, "true");
                }
                connection.getChildren().add(label(reference, "HEAD")); ③
                connection.getChildren().add(label(opposite, "TAIL"));
                done.add(opposite);
            }
            diagram.getChildren().add(connection);
        }
    }
```

```
}

private Label label(EReference reference, String placement) {
    Label label = factory.createLabel();
    label.setId(reference.getName());
    label.setText(reference.getName() + (reference.isMany() ? " *" : ""));
    label.getSemanticElements().add(reference);
    label.getAttributes().put(PLACEMENT, placement);
    return label;
}
```

① References connect the class nodes themselves.

② A reference without an opposite is labeled at its center.

③ A reference with an opposite is labeled at the end it leads to, its head, and the opposite at the other end, its tail. ELK's option `elk.edgeLabels.placement` on each label says which.

Containment is an attribute of the connection, which the renderers draw as a diamond at the containing end.

5.5. The package diagram

Listing 11. The package diagram

```
/**
 * The classes of a package, and the classes of other packages they extend or refer to.
 */
public static Diagram packageDiagram(EPackage ePackage) {
    String name = ePackage.getName();
    EcoreDiagrams diagrams = new EcoreDiagrams(name, name + " package");
    Set<EClass> members = new LinkedHashSet<>();
    for (EClassifier classifier: ePackage.getEClassifiers()) {
        if (classifier instanceof EClass eClass) {
            members.add(eClass);
        }
    }
    Set<EClass> all = new LinkedHashSet<>(members);
    for (EClass member: members) {
        all.addAll(member.getESuperTypes());
        for (EReference reference: member.getEReferences()) {
            all.add(reference.getEReferenceType());
        }
    }
    diagrams.classes(all);
    diagrams.generalizations();
    diagrams.references(members);
    return diagrams.diagram;
}
```

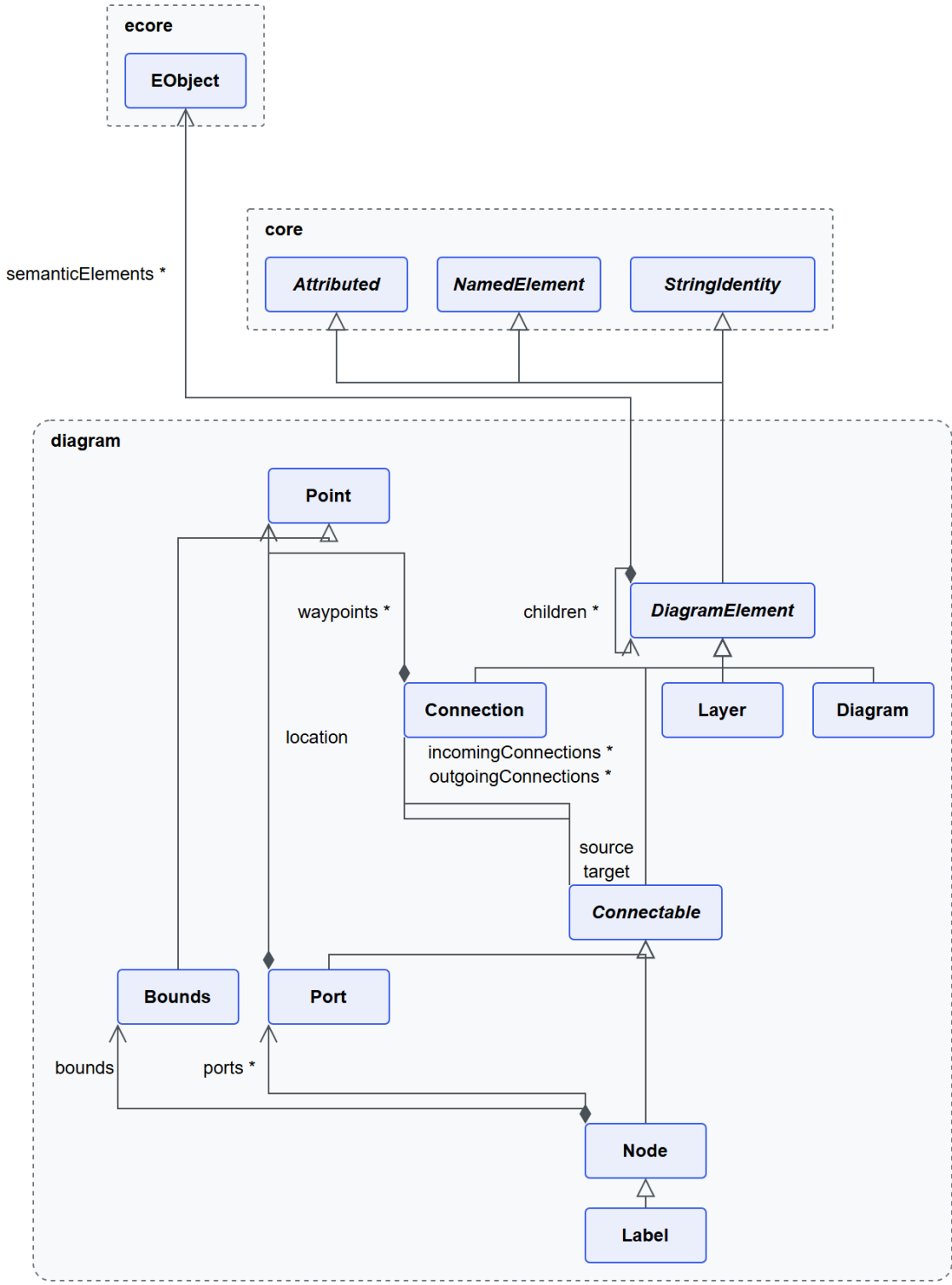


Figure 1. The diagram package, generated, laid out by ELK, and rendered by Draw.io's viewer

The ten classes of the package are in the *diagram* container; the three classes of the runtime's core model it uses, in *core*; and `EObject`, the type of `semanticElements`, in *ecore*. Generalizations rise from the subclasses into shared ports, and the two pairs of opposites, `source` and `outgoingConnections`, `target` and `incomingConnections`, are one connection each, labeled at both ends.

5.6. The class context diagram

Listing 12. The class context diagram

```
/**
 * A class, its supertypes, its direct subtypes in the class's package, the classes its
 * references lead to, and the classes in its package whose references lead to it.
 */
public static Diagram classContext(EClass eClass) {
    EcoreDiagrams diagrams = new EcoreDiagrams(eClass.getName(), eClass.getName() + " context");
    Set<EClass> all = new LinkedHashSet<>();
    all.add(eClass);
    all.addAll(eClass.getEAllSuperTypes());
    Set<EClass> referrers = new LinkedHashSet<>();
    for (EClassifier classifier: eClass.getEPackage().getEClassifiers()) {
        if (classifier instanceof EClass other) {
            if (other.getESuperTypes().contains(eClass)) {
                all.add(other); // A direct subtype
            }
            for (EReference reference: other.getEReferences()) {
                if (reference.getEReferenceType() == eClass) {
                    referrers.add(other);
                }
            }
        }
    }
    all.addAll(referrers);
    Set<EClass> sources = new LinkedHashSet<>(referrers);
    sources.add(eClass);
    for (EReference reference: eClass.getEAllReferences()) {
        all.add(reference.getEReferenceType());
        sources.add(reference.getEContainingClass());
    }
    diagrams.classes(all);
    diagrams.generalizations();
    diagrams.references(sources, reference -> reference.getEContainingClass() != eClass
            && !eClass.getEAllSuperTypes().contains(reference.getEContainingClass())
            && reference.getEReferenceType() != eClass);
    return diagrams.diagram;
}
```

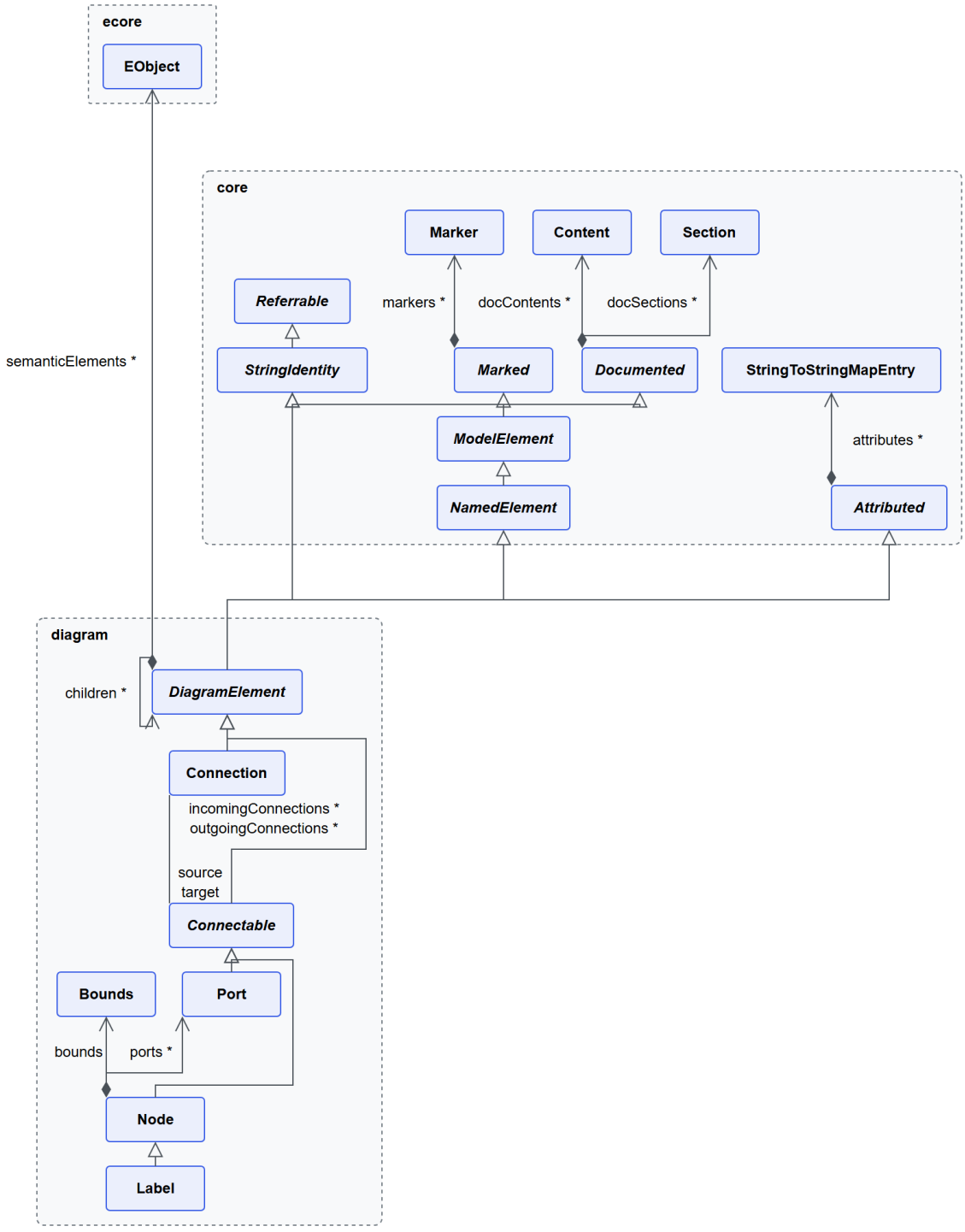


Figure 2. The context of the class Node

The context of `Node` follows its supertypes up through `Connectable` and `DiagramElement` into the core

model, where `ModelElement` merges three supertypes into one line. Its own references lead to `Bounds` and `Port`; references it inherits are drawn from the class that declares them, so `children` loops on `DiagramElement`, and `Connectable`'s `connections` lead to `Connection`.

5.7. Checking the layout

The tests check the conventions on the laid out diagrams, not only their structure:

Listing 13. Generating and laying out the diagrams (`EcoreDiagramTests.java`)

```
packageDiagram = EcoreDiagrams.packageDiagram(DiagramPackage.eINSTANCE);
new ElkLayout().layout(packageDiagram);

nodeContext = EcoreDiagrams.classContext(DiagramPackage.Literals.NODE);
new ElkLayout().layout(nodeContext);

DiagramSite.write(packageDiagram, "diagram-package"); ①
DiagramSite.write(nodeContext, "node-context");
DiagramSite.index(List.of(
        new DiagramSite.Entry("diagram-package", "Diagram package",
                "the classes of the diagram model and what they extend and refer to"),
        new DiagramSite.Entry("node-context", "Node context",
                "the class Node, its supertypes, subtypes, references and referrers")));
```

① Writes the React Flow page and the Draw.io document of each diagram (Chapter 6).

Listing 14. Generalizations: from port to port, rectilinear, upward

```
for (Diagram diagram: List.of(packageDiagram, nodeContext)) {
    for (Connection connection: EcoreDiagrams.connections(diagram)) {
        if (GENERALIZATION.equals(connection.getAttributes().get(KIND))) {
            Port top = (Port) connection.getSource();
            Port bottom = (Port) connection.getTarget();
            assertThat(top.getLocation().getY()).isZero(); ①
            assertThat(bottom.getLocation().getY()).isEqualTo(1);
            List<double[]> route = Coordinates.route(connection);
            assertThat(route.getFirst()).containsExactly(Coordinates.port(top)); ②
            assertThat(route.getLast()).containsExactly(Coordinates.port(bottom));
            assertRectilinear(route); ③
            assertThat(route.getLast()[1]).isLessThan(route.getFirst()[1]); ④
        }
    }
}
```

① From the top of the subclass to the bottom of the superclass.

② The route starts and ends exactly at the ports.

③ Every segment is horizontal or vertical.

④ The superclass is above.

Listing 15. References: between nodes, opposites grouped, labels at the right ends

```
List<Connection> references = new ArrayList<>();
for (Connection connection: EcoreDiagrams.connections(packageDiagram)) {
    if (REFERENCE.equals(connection.getAttributes().get(KIND))) {
        assertThat(connection.getSource()).isInstanceOf(Node.class); ①
        assertThat(connection.getTarget()).isInstanceOf(Node.class);
        references.add(connection);
    }
}
Connection incoming = references.stream()
    .filter(c -> c.getSemanticElements().contains(CONNECTABLE__INCOMING_CONNECTIONS))
    .findFirst()
    .orElseThrow();
assertThat(incoming.getSemanticElements()) ②
    .containsExactly(CONNECTABLE__INCOMING_CONNECTIONS, CONNECTION__TARGET);
assertThat(incoming.getChildren()).hasSize(2);
Label head = (Label) incoming.getChildren().get(0);
Label tail = (Label) incoming.getChildren().get(1);
assertThat(head.getText()).isEqualTo("incomingConnections *");
assertThat(tail.getText()).isEqualTo("target");

List<double[]> route = Coordinates.route(incoming);
double[] frame = Coordinates.frame(incoming);
double[] start = route.getFirst();
double[] end = route.getLast();
assertThat(distance(head, frame, end)).isLessThan(distance(head, frame, start)); ③
assertThat(distance(tail, frame, start)).isLessThan(distance(tail, frame, end));

long opposites = references.stream()
    .filter(c -> c.getSemanticElements().size() == 2)
    .count();
long declared = DiagramPackage.eINSTANCE.getEClassifiers().stream()
    .filter(EClass.class::isInstance)
    .flatMap(c -> ((EClass) c).getEReferences().stream())
    .filter(r -> r.getEOpposite() != null)
    .count();
assertThat(opposites * 2).isEqualTo(declared); ④
```

① References attach to nodes.

② The connection depicts both references of the pair.

③ The head label is closer to the end of the route, the tail label to its start.

④ Every pair of opposites in the package is one connection.

Listing 16. What the context of Node contains

```
List<String> classes = EcoreDiagrams.nodes(nodeContext).stream()
    .filter(n -> CLASS.equals(n.getAttributes().get(KIND)))
    .map(Node::getName)
    .toList();
assertThat(classes).contains(
        "Node", "Connectable", "DiagramElement", "NamedElement", // the class and its supertypes
        "Label", // a subtype
        "Bounds", "Port", // what its references lead to
        "Connection"); // what an inherited reference leads to, and refers back
assertThat(classes).doesNotContain("Diagram", "Layer"); ①
```

① Siblings that the context does not need, `Diagram` and `Layer`, are left out.

Chapter 6. Rendering with React Flow and Draw.io

A laid out diagram is geometry: bounds, routes and label positions. The demos render it twice, with React Flow in the browser and with Draw.io, and both show the layout as ELK computed it.

6.1. React Flow

React Flow draws nodes and edges in the browser. By default it routes edges itself, between handles on the nodes; here it should not, because ELK already routed them. So the edges are of a custom type that draws ELK's route and labels as they are.

The renderer builds a React Flow model, which the runtime's React generator turns into a JavaScript module, with no build step and no npm:

Listing 17. The flow (`ReactFlowRenderer.java`)

```
ReactFlow flow = FLOW.createReactFlow();
flow.setNodeTypes(nodeTypes);
flow.setEdgeTypes(edgeTypes);
flow.setFitView(true);
flow.setNodesDraggable(false); ①
flow.setNodesConnectable(false);
flow.setControls(FLOW.createControls());
flow.setBackground(FLOW.createBackground());
for (DiagramElement child: diagram.getChildren()) {
    if (child instanceof Node node) {
        addNode(node, null, flow);
    }
}
diagram.eAllContents().forEachRemaining(e -> {
    if (e instanceof Connection connection) {
        flow.getEdges().add(edge(connection));
    }
});
```

① Nodes cannot be moved: the edges are drawn from fixed routes, which would not follow them.

Listing 18. Nodes, nested as the diagram nests them

```
private void addNode(Node node, FlowNode parent, ReactFlow flow) {
    FlowNode flowNode = FLOW.createFlowNode();
```

```
    flowNode.setId(id(node));
    boolean container = node.getChildren().stream().anyMatch(Node.class::isInstance);
    flowNode.setCustomType(container ? "packageNode" : "classNode");
    Bounds bounds = node.getBounds();
    XYPosition position = FLOW.createXYPosition();
    position.setX(bounds.getX()); ①
    position.setY(bounds.getY());
    flowNode.setPosition(position);
    flowNode.setWidth(bounds.getWidth());
    flowNode.setHeight(bounds.getHeight());
    flowNode.setParent(parent); ②
    FlowData data = FLOW.createFlowData();
    data.setLabel(node.getName());
    if ("true".equals(node.getAttributes().get(EcoreDiagrams.ABSTRACT))) {
        data.getValues().add(expression("abstract", "true"));
    }
    flowNode.setData(data);
    flow.getNodes().add(flowNode); // Parents come before their children, as React Flow requires
    flowNodes.put(node, flowNode);
    for (DiagramElement child: node.getChildren()) {
        if (child instanceof Node childNode) {
            addNode(childNode, flowNode, flow);
        }
    }
}
```

① React Flow positions child nodes relative to their parent, as the diagram model does, so bounds carry over as they are.

② The parent node, for packages, comes before its children.

Listing 19. Edges carry their route and labels as data

```
private FlowEdge edge(Connection connection) {
    FlowEdge edge = FLOW.createFlowEdge();
    edge.setId(connection.getId());
    edge.setSource(flowNodes.get(node(connection.getSource())));
    edge.setTarget(flowNodes.get(node(connection.getTarget())));
    edge.setCustomType("elk"); ①
    FlowData data = FLOW.createFlowData();
    data.getValues().add(expression("points", points(Coordinates.route(connection)))); ②
    data.getValues().add(expression("labels", labels(connection)));
    Map<String, String> attributes = connection.getAttributes().map();
    String kind = attributes.get(EcoreDiagrams.KIND);
    data.getValues().add(value("kind", kind));
    boolean containment = "true".equals(attributes.get(EcoreDiagrams.CONTAINMENT));
    boolean container = "true".equals(attributes.get(EcoreDiagrams.CONTAINER));
    boolean bidirectional = connection.getChildren().size() > 1;
    if (containment) {
        data.getValues().add(value("markerStart", "diamond"));
    }
    if (EcoreDiagrams.GENERALIZATION.equals(kind)) {
```

```
        data.getValues().add(value("markerEnd", "triangle"));
    } else if (container) {
        data.getValues().add(value("markerEnd", "diamond"));
    } else if (!bidirectional) {
        data.getValues().add(value("markerEnd", "arrow"));
    }
    edge.setData(data);
    return edge;
}
```

① The custom edge type.

② The route, in absolute coordinates, as a JavaScript array literal.

The edge component is JavaScript in a code block of the model, with the imports it needs declared as references:

Listing 20. The custom edge, drawing ELK's route

```
/**
 * Draws the route ELK computed, with markers, and the labels where ELK put them.
 */
private static FunctionDeclaration elkEdge() {
    return function("ElkEdge", """
            const { id, data } = props;
            const path = data.points
              .map((point, index) => (index === 0 ? "M " : "L ") + point.x + " " + point.y)
              .join(" ");
            const marker = name => (name ? "url(#" + name + ")" : undefined);
            return createElement(Fragment, null,
              createElement(BaseEdge, {
                id,
                path,
                className: data.kind,
                markerStart: marker(data.markerStart),
                markerEnd: marker(data.markerEnd)
              }),
              createElement(EdgeLabelRenderer, null, data.labels.map(label =>
                createElement("div", {
                  key: label.id,
                  className: "elk-label nodrag nopan",
                  style: {
                    transform: "translate(" + label.x + "px, " + label.y + "px)",
                    width: label.width + "px",
                    height: label.height + "px"
                  }
                }, label.text))));""",
            external("createElement", "react"),
            external("Fragment", "react"),
            external("BaseEdge", XYFLOW),
            external("EdgeLabelRenderer", XYFLOW));
```

```
}
```

The markers, a hollow triangle for generalizations, an open arrow for references and a filled diamond for containment, are SVG markers defined once in the page. The nodes have handles, which React Flow needs to attach edges, but hidden, since the edges do not use them.

6.2. Draw.io

The Draw.io renderer writes a Draw.io document with the runtime's Draw.io API:

Listing 21. Nodes, with ports as connection points (`DrawioRenderer.java`)

```
private void addNode(Node node, Layer<?> parent) {
    org.nasdanika.sdk.runtime.drawio.Node drawioNode = parent.createNode(); ①
    drawioNode.setId(node.getId());
    drawioNode.setLabel(node.getName());
    Bounds bounds = node.getBounds();
    drawioNode.getGeometry().setBounds(
            bounds.getX(),
            bounds.getY(),
            bounds.getWidth(),
            bounds.getHeight());
    boolean container = node.getChildren().stream().anyMatch(Node.class::isInstance);
    boolean isAbstract = "true".equals(node.getAttributes().get(EcoreDiagrams.ABSTRACT));
    String fontStyle = "fontStyle=" + (isAbstract ? 3 : 1) + ";"; // 1 bold, 3 bold italic
    String style = container ? PACKAGE_STYLE : CLASS_STYLE + fontStyle;
    drawioNode.getStyle().putAll(Util.loadStyle(style));
    nodes.put(node, drawioNode);
    for (Port port: node.getPorts()) {
        ports.put(port, drawioNode.createConnectionPoint( ②
                port.getLocation().getX(),
                port.getLocation().getY()));
    }
    for (DiagramElement child: node.getChildren()) {
        if (child instanceof Node childNode) {
            addNode(childNode, drawioNode);
        }
    }
}
```

① A Draw.io node is a layer, so packages contain classes as the diagram does, with bounds relative to the container.

② Ports become Draw.io connection points at the same relative location.

Listing 22. Connections, waypoints and end labels

```
private void addConnection(Connection connection, Layer<?> layer) {
    List<double[]> route = Coordinates.route(connection);
    org.nasdanika.sdk.runtime.drawio.Connection drawioConnection = layer.createConnection(
            end(connection.getSource(), route.getFirst()), ①
            end(connection.getTarget(), route.getLast()));
    drawioConnection.setId(connection.getId());
    for (double[] point: route.subList(1, route.size() - 1)) {
        drawioConnection.getPoints().add(point[0], point[1]); ②
    }
    drawioConnection.getStyle().putAll(Util.loadStyle(EDGE_STYLE + decorations(connection)));

    for (DiagramElement child: connection.getChildren()) {
        if (child instanceof Label label && label.getBounds() != null) {
            addLabel(label, route, Coordinates.frame(connection), drawioConnection);
        }
    }
}

private void addLabel(Label label, List<double[]> route, double[] frame,
        org.nasdanika.sdk.runtime.drawio.Connection drawioConnection) {
    String placement = label.getAttributes().get(EcoreDiagrams.PLACEMENT);
    double position;
    double[] anchor;
    if ("TAIL".equals(placement)) {
        position = -1; ③
        anchor = route.getFirst();
    } else if ("HEAD".equals(placement)) {
        position = 1;
        anchor = route.getLast();
    } else {
        position = 0;
        anchor = Coordinates.middle(route);
    }
    Bounds b = label.getBounds();
    double centerX = frame[0] + b.getX() + b.getWidth() / 2;
    double centerY = frame[1] + b.getY() + b.getHeight() / 2;
    org.nasdanika.sdk.runtime.drawio.Node drawioLabel = drawioConnection.createNode();
    drawioLabel.setLabel(label.getText());
    drawioLabel.getStyle().putAll(Util.loadStyle(LABEL_STYLE));
    Geometry geometry = drawioLabel.getGeometry();
    geometry.setRelative(true);
    geometry.setX(position);
    org.nasdanika.sdk.runtime.drawio.Point offset = geometry.getPoints().add();
    offset.setRole("offset"); ④
    offset.setX(centerX - anchor[0]);
    offset.setY(centerY - anchor[1]);
}
```

① A connection attached to a node gets a connection point where ELK's route touches the node, so Draw.io starts and ends it exactly there.

② The rest of the route becomes Draw.io's waypoints.

③ Draw.io positions an edge label along the edge, from -1 at the source to 1 at the target. A tail label is anchored at the source end, a head label at the target end, a center label at the middle.

④ The offset from the anchor puts the label where ELK placed it. Labels stay attached to their edge when someone moves it in Draw.io.

The document opens in Draw.io as laid out, and is a normal Draw.io document from then on: people can move boxes, restyle them, and add what a generator could not know.

6.3. The site, and serving it

Listing 23. Writing both renderings of a diagram (DiagramSite.java)

```
/**
 * Writes the React Flow page and module, the Draw.io document and its viewer page.
 */
static void write(Diagram diagram, String name) throws Exception {
    String module = new ReactGenerator().generate(ReactFlowRenderer.module(diagram, name));
    write(SITE.resolve("modules/" + name + ".js"), module);
    write(SITE.resolve(name + ".html"), html(reactFlowPage(diagram.getName(), name)));

    Document document = DrawioRenderer.render(diagram);
    write(SITE.resolve(name + ".drawio"), document.save(false));
    write(SITE.resolve(name + "-drawio.html"), html(drawioPage(diagram.getName(), document)));
}
```

The site has a React Flow page, a Draw.io document and a page showing the document in Draw.io's viewer for each diagram, and an index. The runtime's volumes serve it over HTTP:

Listing 24. Serving the site (ServeDiagrams.java)

```
public static HttpServer serve(Path site, int port) throws IOException {
    var address = new InetSocketAddress(InetAddress.getLoopbackAddress(), port);
    HttpServer server = HttpServer.create(address, 0);
    server.setExecutor(Executors.newVirtualThreadPerTaskExecutor());
    server.createContext("/diagrams", new VolumeHttpHandler(Volume.of(site))); ①
    server.start();
    return server;
}
```

① A volume over the site's directory, served by the runtime's volume HTTP handler, which sends JavaScript modules with the media type browsers require for them.

To see the diagrams, build the module's tests, which write the site into `models/diagram/elk/target/diagrams`, then run `ServeDiagrams` from an IDE or with the test class path, and open the URL it prints, `127.0.0.1:8080/diagrams/index.html`. The pages load React, React Flow and Draw.io's viewer from content delivery networks.

The tests check both renderings, and that the server serves them:

Listing 25. Checking the renderings (`EcoreDiagramTests.java`)

```
String module = Files.readString(DiagramSite.SITE.resolve("modules/diagram-package.js"));
assertThat(module)
    .contains("function ElkEdge(props)")
    .contains("edgeTypes")
    .contains("type: \"elk\"")
    .contains("markerEnd: \"triangle\"");

File file = DiagramSite.SITE.resolve("diagram-package.drawio").toFile();
Document drawio = Document.load(file); ①
var node = (org.nasdanika.sdk.runtime.drawio.Node) drawio.getModelElementById("Node");
Bounds bounds = EcoreDiagrams.nodes(packageDiagram).stream()
    .filter(n -> "Node".equals(n.getId()))
    .findFirst()
    .orElseThrow()
    .getBounds();
assertThat(node.getGeometry().getX()).isEqualTo(bounds.getX());
assertThat(node.getGeometry().getWidth()).isEqualTo(bounds.getWidth());
```

① The Draw.io document, read back with the Draw.io API, has the bounds ELK computed.

Listing 26. Checking the server

```
HttpServer server = ServeDiagrams.serve(DiagramSite.SITE, 0); ①
try {
    String root = "http://127.0.0.1:" + server.getAddress().getPort() + "/diagrams/";
    HttpClient client = HttpClient.newHttpClient();
    HttpResponse<String> page = client.send(
            HttpRequest.newBuilder(URI.create(root + "node-context.html")).build(),
            HttpResponse.BodyHandlers.ofString());
    assertThat(page.statusCode()).isEqualTo(200);
    assertThat(page.body()).contains("./modules/node-context.js");
    HttpResponse<String> module = client.send(
            HttpRequest.newBuilder(URI.create(root + "modules/node-context.js")).build(),
            HttpResponse.BodyHandlers.ofString());
    assertThat(module.headers().firstValue("Content-Type")).hasValueSatisfying(
            type -> assertThat(type).startsWith("text/javascript")); ②
} finally {
    server.stop(0);
}
```

① Port 0 picks a free port.

② Browsers load modules only with a JavaScript media type.

Chapter 7. Figures of generated diagrams

The figures of this book are the diagrams the tests generate, which raises a question every generated diagram in documentation raises: who renders the picture? A diagram drawn in Draw.io is rendered by Draw.io when someone saves it. A diagram written by a program has no picture until something renders it.

The figures here were rendered with the runtime's figure tool, `doc/zephyr/tools/drawio-figure`: it opens a Draw.io document in Draw.io's own viewer in a headless browser, takes a screenshot at twice the size, crops it to the diagram, and writes a PNG image with the document embedded, as Draw.io does when it exports a diagram with "Include a copy of my diagram". So each figure is also a Draw.io document, which Draw.io opens for editing and the Draw.io API loads.

A test compares the documents in the figures with a fresh layout, so a change to the generator or to the adapter fails the build until the figures are rendered again:

Listing 27. Checking a figure against a fresh layout (`FigureTests.java`)

```
static void assertMatches(Diagram diagram, String figure) throws Exception {
    URL png = FigureTests.class.getResource(figure);
    Document document = Document.loadFromPngMetadata(png).getFirst(); ①
    for (Node node: EcoreDiagrams.nodes(diagram)) {
        Geometry drawn = geometry(document, node.getId());
        if (EcoreDiagrams.CLASS.equals(node.getAttributes().get(EcoreDiagrams.KIND))) {
            Bounds bounds = node.getBounds(); ②
            assertThat(drawn.getWidth()).as(node.getId()).isEqualTo(bounds.getWidth());
            assertThat(drawn.getHeight()).as(node.getId()).isEqualTo(bounds.getHeight());
        }
    }
    for (Connection connection: EcoreDiagrams.connections(diagram)) {
        assertThat(document.getModelElementById(connection.getId())) ③
            .as(connection.getId() + " in " + figure)
            .isInstanceOf(org.nasdanika.sdk.runtime.drawio.Connection.class);
    }
}
```

① The Draw.io document embedded in the figure.

② Classes are sized by their names, so their sizes must match exactly.

③ Every connection of the fresh layout is in the figure.

Positions are not compared.



ELK's layout of the generated diagrams is the same within a JVM, but has differed slightly between runs: the height of the *diagram* container was 560 in some runs and 570 in others, and its x coordinate in the context diagram 48 and 48.5. Repeated layouts in one run were always identical. This is consistent with iteration over hash-based collections inside ELK, which `elk.randomSeed` does not control. That is why the figure test compares structure and sizes, not positions. Find the cause, and if a deterministic layout is possible, compare positions too: a figure that matches the build to the pixel is the point of generated figures.

The figure tool is a script with Python, Chrome and a JDK, run by hand. When the runtime can render diagrams in its build, with Playwright as planned for narrated videos, the figures should be rendered by the build and this test can go.

Part III: Looking Ahead

What comes after the adapter.

Chapter 8. What comes next

Everything in this chapter is **PLANNED**.

8.1. Renderers as modules

The React Flow and Draw.io renderers of Chapter 6 are test code, written for the demos. Both are general enough to become modules: diagram model to React Flow, with the custom edge that draws a computed route, and diagram model to Draw.io. The Draw.io one is the generator half of the Draw.io model's loader and generator, being migrated alongside the diagram model.

8.2. Laying out what people drew

A diagram loaded from Draw.io has positions already. Laying it out from scratch throws away the work of the person who drew it. ELK's layered algorithm has interactive strategies that keep the existing order of nodes, and options that respect a node's current position; with them, a layout can make room for new elements and tidy routes while keeping what people decided. That is the layout step of merging a regenerated diagram with the edits people made to the previous one.

8.3. Layout in the browser

elkjs runs the same algorithms in JavaScript. The adapter could write ELK's JSON format, so that a diagram is laid out in the browser, where a person can collapse a container or filter a layer and see the diagram laid out again without a server.

8.4. A layout service, when there is a choice

When a second layout engine has a real consumer, `ElkLayout.layout(DiagramElement)` becomes an interface, and ELK one of its providers, found through the runtime's capability framework with a requirement that names the engine and its options (Chapter 2).

8.5. Labels measured, not estimated

Label and node sizes are estimated from the number of characters. A measurer with real font metrics, from Java 2D, or from the same browser that renders the React Flow page, would make labels fit exactly.

8.6. Diagrams in narrated videos

Diagrams generated and laid out this way are meant to be scenes of the runtime's narrated videos: a module graph or a class diagram on screen, with elements highlighted as the narration mentions them, regenerated when the code changes.

Appendix A: Layout options used most

ELK's options are documented in full at eclipse.dev/elk, under *Reference*. These are the ones a diagram is most likely to set, as attribute keys and values. Enumeration values are written in capitals, numbers as numbers, and vectors as `(x, y)`.

Key	Values	Effect
`elk.algorithm`	`layered`, `mrtree`, `force`, `radial`	The algorithm. Layered is the default, for flows and directed graphs.
`elk.direction`	`UP`, `DOWN`, `LEFT`, `RIGHT`	The direction edges flow in, for layered and tree layouts.
`elk.edgeRouting`	`ORTHOGONAL`, `POLYLINE`, `SPLINES`	How edges are drawn: rectilinear, straight segments, or curves.
`elk.hierarchyHandling`	`INCLUDE_CHILDREN`, `SEPARATE_CHILDREN`	Whether nested nodes are laid out together with their containers. The adapter sets `INCLUDE_CHILDREN` when nodes are nested.
`elk.spacing.nodeNode`	A number	Space between nodes in the same layer.
`elk.layered.spacing.nodeNodeBetweenLayers`	A number	Space between layers.
`elk.spacing.edgeLabel`	A number	Space between an edge and its labels.
`elk.layered.mergeEdges`	`true`, `false`	Edges into the same port share segments.
`elk.edgeLabels.placement`	`HEAD`, `TAIL`, `CENTER`	On a label: which part of the edge it goes with.
`elk.portConstraints`	`FREE`, `FIXED_SIDE`, `FIXED_ORDER`, `FIXED_POS`	On a node: how much ELK may move its ports. The adapter sets `FIXED_POS` on nodes with ports.
`elk.port.side`	`NORTH`, `SOUTH`, `EAST`, `WEST`	On a port: its side. The adapter sets it from the port's location.
`elk.nodeSize.minimum`	`(width, height)`	On a node: its minimum size.

Key	Values	Effect
`elk.padding`	`[top=10, left=10, bottom=10, right=10]`	On a container: room around its children.
`elk.randomSeed`	A number	The seed of algorithms that use randomness.

Appendix B: Building this book, and its figures

This book is AsciiDoc in `models/diagram/elk/src/docs/asciidoc`, built with the module after its tests have passed:

```
mvn -pl models/diagram/elk -am verify                          # the module, its tests, this book
mvn -pl models/diagram/elk verify -Dzephyr.editor-notes=false    # without editor's notes
```

The PDF and the single-page HTML are written to `target/book` and attached to the module's artifact with the classifier `book`; the test sources are attached with the classifier `test-sources`.

The tests write the demo site to `target/diagrams`. To render the figures again after a change, from the runtime's root:

```
tools=doc/zephyr/tools/drawio-figure
figures=models/diagram/elk/src/test/resources/org/nasdanika/sdk/runtime/models/diagram/elk/tests
$tools/render.sh models/diagram/elk/target/diagrams/diagram-package.drawio $figures/diagram-
package.png
$tools/render.sh models/diagram/elk/target/diagrams/node-context.drawio $figures/node-context.png
```

The tool needs Python, a JDK, Chrome or Chromium, and internet access for Draw.io's viewer.

B.1. The tests run on the module path

The tests are patched into the module on the module path, and use modules the module does not require: the React Flow, React, TypeScript, HTML and Bootstrap models, volumes, the Draw.io API, and the JDK's XML, desktop and HTTP modules. The module's POM adds them for test compilation and for the test run with `--add-modules` and `--add-reads`. `--add-reads` does not carry the readability that `requires transitive` implies, so the models behind React Flow are listed one by one.

Appendix C: Glossary

Automatic module

A jar without a module declaration on the module path, named by its manifest or its file name. ELK's jars are automatic modules.

Container

A node that contains nodes. In the class diagrams, a package.

Generalization

A connection from a subclass to a superclass.

Head, tail

The target end and the source end of a connection, where end labels go.

Layered layout

ELK's algorithm for directed graphs: nodes in layers along the direction of the edges, few crossings, and orthogonal or other routes between layers.

Layout option

A named setting of ELK, such as `elk.direction`, set on the graph or on a node, port, edge or label. In the diagram model, an attribute whose key starts with `elk.`.

Port

An attachment point on a node, at a location relative to the node's size.

Route

The points a connection passes through, from its source to its target.

Text measurer

What estimates the size of text, for labels and for nodes without bounds.